

18-661 Introduction to Machine Learning

Overfitting and the Bias-Variance Trade-off

Spring 2026

ECE – Carnegie Mellon University

Announcements

- HW 1 is due on Friday, Jan 30th. Note that you can use up to 2 late days per homework, and up to 5 during the semester.
- First mini-exam is on Feb 9th, during the first half of the class. You are allowed to bring 1 one-sided handwritten US-letter-sized cheat sheet. No electronic devices are permitted. Calculators are allowed but will not be necessary.

1. Review of Ridge Regression
2. Non-linear Basis Functions
3. Overfitting and Regularization
4. Hyperparameter Tuning and Cross-Validation
5. Bias-Variance Trade-off

Review of Ridge Regression

What can go wrong with the LMS solution?

$$\mathbf{w}^{LMS} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Why might $\mathbf{X}^\top \mathbf{X}$ be non-invertible?

- **Answer 1:** $N < D$. Not enough data to estimate all parameters.
- **Answer 2:** Columns of $\mathbf{X}$ are not linearly independent, e.g., some features are linear functions of other features. In this case, solution is not unique. Examples:
 - A feature is a re-scaled version of another, for example, having two features correspond to length in meters and feet respectively
 - Same feature is repeated twice – could happen when there are many features
 - A feature has the same value for all data points
 - Sum of two features is equal to a third feature

Ridge regression

Intuition: what does a non-invertible $\mathbf{X}^\top \mathbf{X}$ mean?

Consider the EVD of this matrix:

$$\mathbf{X}^\top \mathbf{X} = \mathbf{V} \begin{bmatrix} \lambda_1 & 0 & 0 & \cdots & 0 \\ 0 & \lambda_2 & 0 & \cdots & 0 \\ 0 & \cdots & \cdots & \cdots & 0 \\ 0 & \cdots & \cdots & \lambda_r & 0 \\ 0 & \cdots & \cdots & 0 & 0 \end{bmatrix} \mathbf{V}^\top$$

where $\lambda_1 \geq \lambda_2 \geq \cdots \lambda_r > 0$, $r < D$, and $\mathbf{V}$ is a unitary matrix (its transpose is its inverse). We will need to divide by zero to compute $(\mathbf{X}^\top \mathbf{X})^{-1} \dots$

Fix the problem: ensure all singular values are non-zero:

$$\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I} = \mathbf{V} \text{diag}(\lambda_1 + \lambda, \lambda_2 + \lambda, \cdots, \lambda) \mathbf{V}^\top$$

where $\lambda > 0$ and $\mathbf{I}$ is the identity matrix.

Regularized least squares (ridge regression)

Solution

$$\mathbf{w} = \left(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I} \right)^{-1} \mathbf{X}^\top \mathbf{y}$$

This is equivalent to adding an extra term to $RSS(\mathbf{w})$

$$\overbrace{\frac{1}{2} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2}^{RSS(\mathbf{w})} + \underbrace{\frac{1}{2} \lambda \|\mathbf{w}\|_2^2}_{\text{regularization}}$$

Benefits

- Numerically more stable, invertible matrix
- Force $\mathbf{w}$ to be small
- Prevent overfitting — more on this later

Non-linear Basis Functions

Is a linear modeling assumption always a good idea?

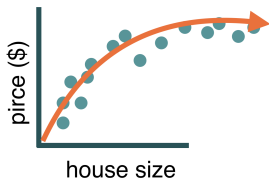


Figure 1: Sale price can saturate as sq.footage increases

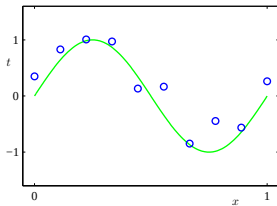


Figure 2: Temperature has cyclic variations over each year

General nonlinear basis functions

We can use a nonlinear mapping:

$$\phi(\mathbf{x}) : \mathbf{x} \in \mathbb{R}^D \rightarrow \mathbf{z} \in \mathbb{R}^M$$

- M is dimensionality of new features $\mathbf{z}$ (or $\phi(\mathbf{x})$)
- M could be greater than, less than, or equal to D

We can apply existing learning methods on the transformed data:

- linear methods: prediction is based on $\mathbf{w}^\top \phi(\mathbf{x})$
- other methods: nearest neighbors, decision trees, etc

Residual sum of squares

$$\frac{1}{2} \sum_n [\mathbf{w}^\top \phi(\mathbf{x}_n) - y_n]^2$$

where $\mathbf{w} \in \mathbb{R}^M$, the same dimensionality as the transformed features $\phi(\mathbf{x})$.

The LMS solution can be formulated with the new design matrix

$$\Phi = \begin{pmatrix} \phi(\mathbf{x}_1)^\top \\ \phi(\mathbf{x}_2)^\top \\ \vdots \\ \phi(\mathbf{x}_N)^\top \end{pmatrix} \in \mathbb{R}^{N \times M}, \quad \mathbf{w}^{\text{LMS}} = \left(\Phi^\top \Phi \right)^{-1} \Phi^\top \mathbf{y}$$

Example: Lots of flexibility in designing features!

x_1 , front (100ft)	x_2 depth (100ft)	$10x_1x_2$, Lot (1k sqft)	Price (100k)
0.5	0.5	2.5	2
0.5	1	5	3.5
0.8	1.5	12	3
1.0	1.5	15	4.5

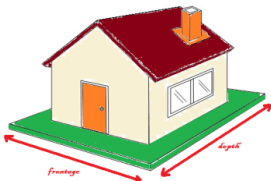


Figure 3: Instead of having frontage and depth as two separate features, it may be better to consider the lot-area, which is equal to $\text{frontage} \times \text{depth}$

Concept Check: How and which features to add?

Suppose you have the original feature vector: $[x_1, x_2]$. Is it okay to transform it to:

- $\phi(\mathbf{x}) = [x_1, x_2, x_1^2, x_2^2]$? YES
- $\phi(\mathbf{x}) = [x_1, x_2, x_1 + x_2]$? NO, linearly dependent features
- $\phi(\mathbf{x}) = [x_1 + x_2, x_1^2, x_2^2]$? YES

Feature design is somewhat of an art and it requires domain knowledge and a good understanding of the dataset

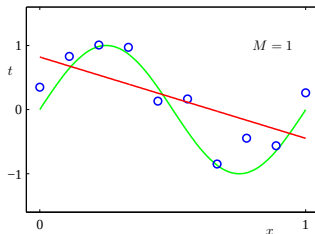
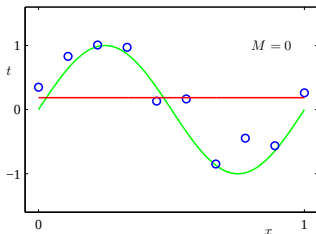
Overfitting and Regularization

Non-linear basis functions: Polynomial regression

Polynomial basis functions

$$\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^M \end{bmatrix} \Rightarrow f(x) = w_0 + \sum_{m=1}^M w_m x^m$$

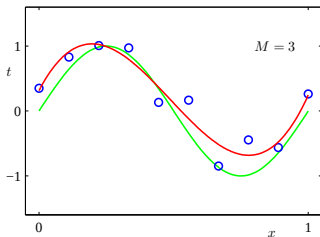
Fitting samples from a sine function:



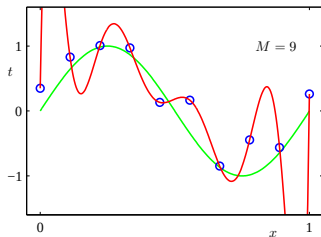
underfitting since $f(x)$ is too simple

Adding high-order terms

$M=3$



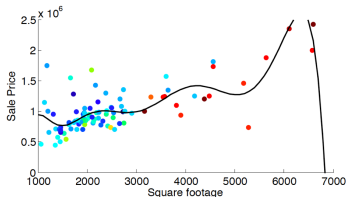
$M=9$: **overfitting**



More complex features lead to better results on the training data, but potentially worse results on new data, e.g., test data!

Overfitting can be quite disastrous

Fitting the housing price data with large M :



Predicted price goes to zero (and is ultimately negative) if you buy a big enough house!

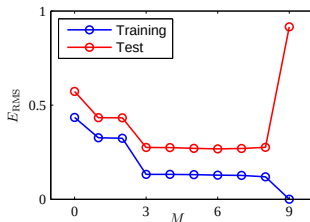
This is called poor generalization/overfitting.

Detecting overfitting

Plot model complexity versus objective function:

- X axis: model complexity, e.g., M
- Y axis: error, e.g., RSS, RMS
(square root of RSS), 0-1 loss

Compute the objective on a training and validation dataset.

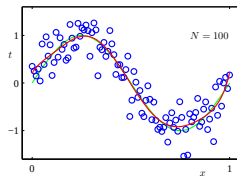
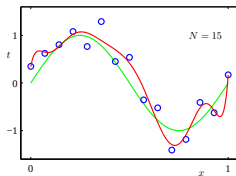
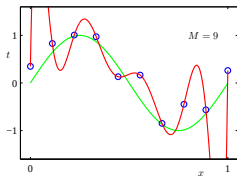


As a model increases in complexity:

- Training error keeps reducing
- Validation error may first reduce but eventually increase

Dealing with overfitting: Option 1

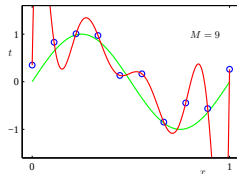
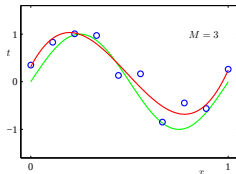
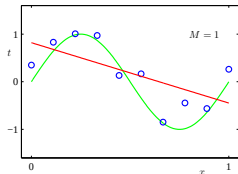
Try to use more training data



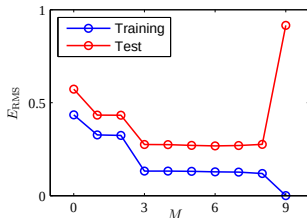
But getting a lot of data can be expensive and time-consuming

Dealing with overfitting: Option 2

Reduce the Number of Features



May not know which and how many features to remove. One idea is to use the training/validation set evaluation plot. $M = 3$ seems to give the best performance.



Dealing with overfitting: Option 3

Regularization Methods: Give preference to ‘simpler’ models

- How do we define a simple linear regression model — $\mathbf{w}^\top \mathbf{x}$?
- Intuitively, the weights corresponding to higher order terms should not be “too large”

	$M = 0$	$M = 1$	$M = 3$	$M = 9$
w_0	0.19	0.82	0.31	0.35
w_1		-1.27	7.99	232.37
w_2			-25.43	-5321.83
w_3			17.37	48568.31
w_4				-231639.30
w_5				640042.26
w_6				-1061800.52
w_7				1042400.18
w_8				-557682.99
w_9				125201.43

Add a term to the objective function.

Choose the parameters to not just minimize risk, but avoid being large.

$$\frac{1}{2} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \frac{1}{2} \lambda \|\mathbf{w}\|_2^2$$

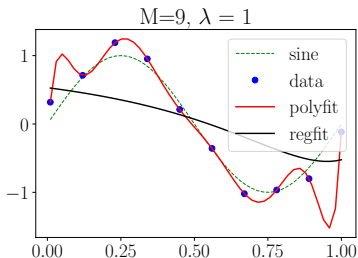
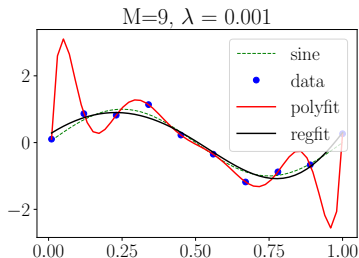
Ridge regression is just regularized linear regression.

Advantages

- Forces the magnitude of $\mathbf{w}$ to be small
- Tries to find a simple model with few parameters
- Generalizes well to new data points

Example: Effect of regularization

- Regularization makes the higher order w_i 's smaller
- Regularized polynomial fit will generalize much better
- As λ increases, the model becomes simpler



Hyperparameter Tuning and Cross-Validation

How should we choose the right amount of regularization?

Can we tune λ on the training dataset?

No: as this will always set λ to zero, i.e., no regularization, defeating our intention of controlling model complexity

λ is thus a **hyperparameter**. To tune it,

- We can use a validation set or do cross validation.
- Pick the value of λ that yields lowest error on the **validation dataset**.

Similar idea applies to tuning learning rate η (or any other hyperparameter) as well.

Tuning by using a validation dataset

Training data are used to learn $f(\cdot)$.

N samples/instances: $\mathcal{D}^{\text{TRAIN}} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\}$

Test data are used to assess the prediction error.

- M samples/instances: $\mathcal{D}^{\text{TEST}} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_M, y_M)\}$
- They are used for assessing how well $f(\cdot)$ will do in predicting an unseen $\mathbf{x} \notin \mathcal{D}^{\text{TRAIN}}$

Validation data are used to optimize hyperparameter(s).

L samples/instances: $\mathcal{D}^{\text{VAL}} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_L, y_L)\}$

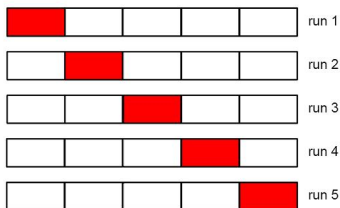
Training data, validation and test data **should not overlap!**

- For each possible value of the hyperparameter (say $\lambda = 1, 3, \dots, 100$)
 - Train a model using $\mathcal{D}^{\text{TRAIN}}$
 - Evaluate the performance of the model on $\mathcal{D}^{\text{VAL}}$
- Choose the model with the best performance on $\mathcal{D}^{\text{VAL}}$
- Evaluate this model on $\mathcal{D}^{\text{TEST}}$ to get the final prediction error

What if we do not have validation data?

- We split the training data into S equal parts.
- We use **each part in turn** as a validation dataset and use the others as a training dataset.
- We choose the hyperparameter such that the model performs the best (based on average, variance, etc.)

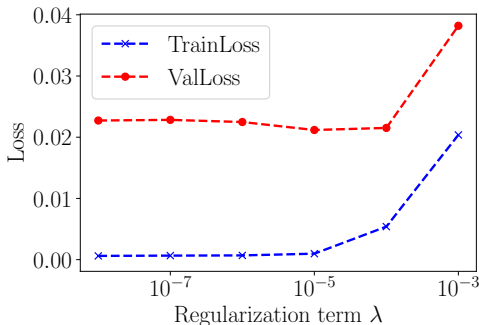
Figure 4: $S = 5$: 5-fold cross validation



Special case: when $S = N$, this will be leave-one-out.

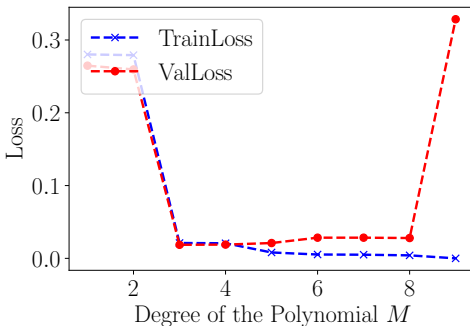
Example: Hyper-parameter Tuning λ

- $\lambda = 10^{-5}$ gives the smallest validation loss
- Strikes a balance between over- and under-fitting



Example: Hyper-parameter Tuning M

- Considering polynomial regression without regularization
- $M = 3$ or $M = 4$ gives the smallest validation loss
- Strikes a balance between over- and under-fitting



Bias-Variance Trade-off

Empirical Risk Minimization

Supervised learning

We aim to build a function $h(\mathbf{x})$ to predict the true value y associated with $\mathbf{x}$. If we make a mistake, we incur a **loss**

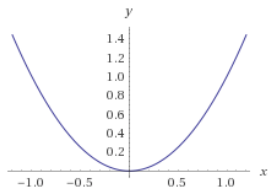
$$\ell(h(\mathbf{x}), y)$$

Example:

Quadratic loss function for regression when y is continuous:

$$\ell(h(\mathbf{x}), y) = [h(\mathbf{x}) - y]^2$$

Ex: when $y = 0$



How good is our predictor?

Risk:

Given the true distribution of data $p(\mathbf{x}, y)$, the **risk** of a given predictor $h(\mathbf{x})$ is its expected loss ℓ :

$$R[h(\mathbf{x})] = \int_{\mathbf{x}, y} \ell(h(\mathbf{x}), y) p(\mathbf{x}, y) d\mathbf{x} dy$$

However, we cannot compute $R[h(\mathbf{x})]$ (we do not know p), so we use the **empirical risk**, given a training dataset $\mathcal{D}$:

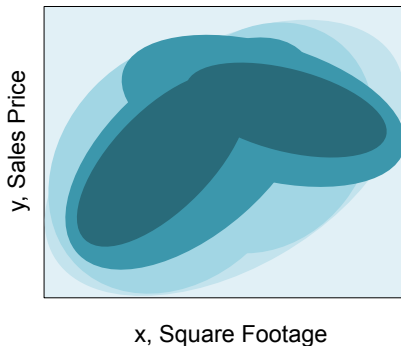
$$R^{\text{EMP}}[h(\mathbf{x})] = \frac{1}{N} \sum_n \ell(h(\mathbf{x}_n), y_n)$$

Intuitively, as $N \rightarrow +\infty$,

$$R^{\text{EMP}}[h(\mathbf{x})] \rightarrow R[h(\mathbf{x})]$$

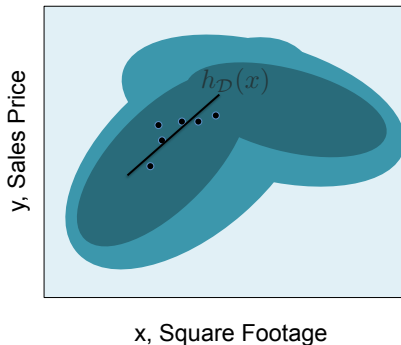
Bias-Variance Trade-off: Illustration

- Joint distribution of square footage x and house sales price y
- Darker color indicates higher probability regions



Bias-Variance Trade-off: Illustration

- We have access to dataset $\mathcal{D}$ sampled from the joint distribution
- Learn linear model $h_{\mathcal{D}}(x)$ based on $\mathcal{D}$



How could this go wrong?

So far, we have been doing **empirical risk minimization (ERM)**

For linear regression, $h(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$, and we use squared loss ℓ .

$$R^{\text{EMP}}[h(\mathbf{x})] = \frac{1}{N} \sum_n \ell(h(\mathbf{x}_n), y_n)$$

$$R[h(\mathbf{x})] = \int_{\mathbf{x}, y} \ell(h(\mathbf{x}), y) p(\mathbf{x}, y) d\mathbf{x} dy$$

What could go wrong with ERM?

- **Limited Data:** We don't know $p(\mathbf{x}, y)$, so we must hope that we have enough training data that the empirical risk approximates the real risk. Otherwise, we will **overfit** to the training data.
- **Limited Function Class:** The function $h(\mathbf{x})$ is restricted to a limited class (e.g. linear functions), which does not allow us to perfectly fit y , even if we had infinitely many training data points.

Bias-Variance Trade-off: Intuition

- **High Bias:** Model is not rich enough to fit the training dataset and achieve low training loss
- **High Variance:** If the training dataset changes slightly, the model changes a lot
- Regularization helps find a middle ground

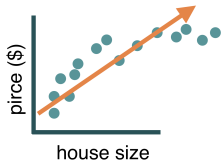


Figure 5: High Bias

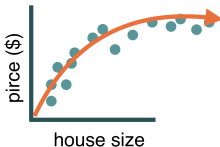


Figure 6: Just Right

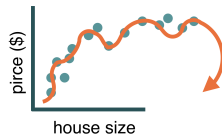


Figure 7: High Variance

Q: Does bias increase/decrease when you increase λ ? **Increase**

Q: Does variance increase/decrease when you increase λ ? **Decrease**

Bias/variance tradeoff for regression

Goal: to understand the sources of prediction errors

- $\mathcal{D}$: our training data
- $h_{\mathcal{D}}(\mathbf{x})$: our prediction function
We are using the subscript $\mathcal{D}$ to indicate that the prediction function is learned on the specific set of training data $\mathcal{D}$
- $\ell(h(\mathbf{x}), y)$: our square loss function for regression

$$\ell(h_{\mathcal{D}}(\mathbf{x}), y) = [h_{\mathcal{D}}(\mathbf{x}) - y]^2$$

- Unknown joint distribution $p(\mathbf{x}, y)$

The effect of finite training samples

Every training sample $\mathcal{D}$ is a sample from the following joint distribution of all possible training datasets

$$\mathcal{D} \sim P(\mathcal{D}) = \prod_{n=1}^N p(\mathbf{x}_n, y_n)$$

Thus, the prediction function $h_{\mathcal{D}}(\mathbf{x})$ is a random function with respect to this distribution of possible training datasets. So is also its risk

$$R[h_{\mathcal{D}}(\mathbf{x})] = \int_{\mathbf{x}} \int_y [h_{\mathcal{D}}(\mathbf{x}) - y]^2 p(\mathbf{x}, y) d\mathbf{x} dy$$

We will now evaluate the **expected risk** $\mathbb{E}_{\mathcal{D}} R[h_{\mathcal{D}}(\mathbf{x})]$: the average risk over the distribution of possible training datasets, $P(\mathcal{D})$.

Bias-Variance Trade-off: Intuition

Error decomposes into 3 terms

$$\mathbb{E}_{\mathcal{D}} R[h_{\mathcal{D}}(\mathbf{x})] = \text{VARIANCE} + \text{BIAS}^2 + \text{NOISE}$$

We will prove this result, and interpret what it means...

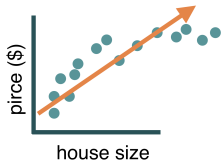


Figure 8: High Bias

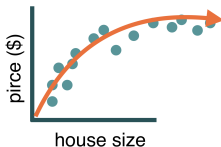


Figure 9: Just Right

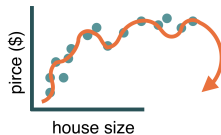


Figure 10: High Variance

Warning: The next few slides are somewhat mathematical – please review them carefully after class.

Average over the distribution of the training data

Expected risk

$$\mathbb{E}_{\mathcal{D}} [R[h_{\mathcal{D}}(\mathbf{x})]] = \int_{\mathcal{D}} \int_{\mathbf{x}} \int_y [h_{\mathcal{D}}(\mathbf{x}) - y]^2 p(\mathbf{x}, y) d\mathbf{x} dy P(\mathcal{D}) d\mathcal{D}$$

Namely, the randomness with respect to $\mathcal{D}$ is marginalized out.

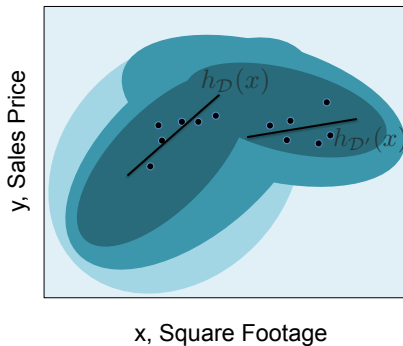
Averaged prediction

$$\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x}) = \int_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x}) P(\mathcal{D}) d\mathcal{D}$$

Namely, if we have seen many training datasets, we predict with the average of the prediction functions learned on each training dataset.

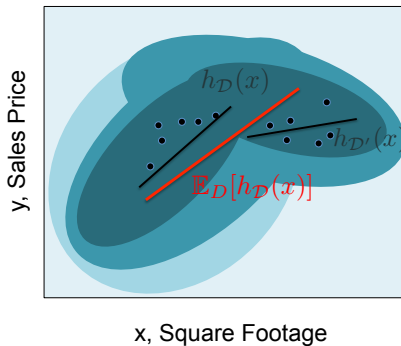
Bias-Variance Trade-off: Illustration

- Learning the model for a different dataset $\mathcal{D}'$ yields a new linear model $h_{\mathcal{D}'}(x)$
- Average of such models over infinitely many datasets sampled from the joint distribution is $\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x})$



Bias-Variance Trade-off: Illustration

- Learning the model for a different dataset $\mathcal{D}'$ yields a new linear model $h_{\mathcal{D}'}(x)$
- Average of such models over infinitely many datasets sampled from the joint distribution is $\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(x)$



We will subtract the averaged prediction from the averaged risk

$$\begin{aligned}\mathbb{E}_{\mathcal{D}} R[h_{\mathcal{D}}(\mathbf{x})] &= \int_{\mathcal{D}} \int_{\mathbf{x}} \int_y [h_{\mathcal{D}}(\mathbf{x}) - y]^2 p(\mathbf{x}, y) d\mathbf{x} dy P(\mathcal{D}) d\mathcal{D} \\&= \int_{\mathcal{D}} \int_{\mathbf{x}} \int_y [h_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x}) \\&\quad + \mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x}) - y]^2 p(\mathbf{x}, y) d\mathbf{x} dy P(\mathcal{D}) d\mathcal{D} \\&= \underbrace{\int_{\mathcal{D}} \int_{\mathbf{x}} \int_y [h_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x})]^2 p(\mathbf{x}, y) d\mathbf{x} dy P(\mathcal{D}) d\mathcal{D}}_{\text{VARIANCE}} \\&\quad + \int_{\mathcal{D}} \int_{\mathbf{x}} \int_y [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x}) - y]^2 p(\mathbf{x}, y) d\mathbf{x} dy P(\mathcal{D}) d\mathcal{D}\end{aligned}$$

Where does the cross-term go?

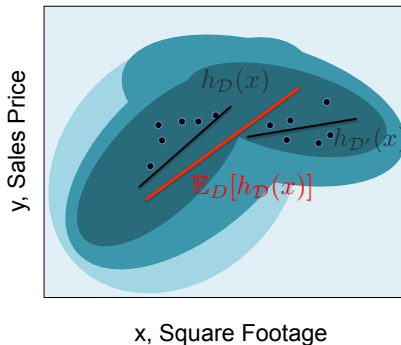
It is zero

$$\begin{aligned} & \int_{\mathcal{D}} \int_{\mathbf{x}} \int_y [h_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x})][\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x}) - y] p(\mathbf{x}, y) d\mathbf{x} dy P(\mathcal{D}) d\mathcal{D} \\ &= \int_{\mathbf{x}} \int_y \left\{ \int_{\mathcal{D}} [h_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x})] P(\mathcal{D}) d\mathcal{D} \right\} [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x}) - y] p(\mathbf{x}, y) d\mathbf{x} dy \\ &= 0 \leftarrow \text{(the integral within the braces vanishes, by definition)} \end{aligned}$$

Bias-Variance Trade-off: Illustration

- Average of such models over infinitely many datasets sampled from the joint distribution is $\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x})$
- Variance term captures how much individual models differ from the average

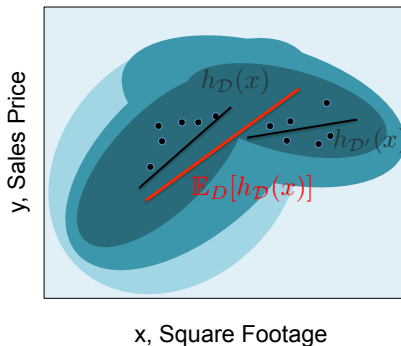
$$\underbrace{\int_{\mathcal{D}} \int_{\mathbf{x}} \int_{\mathbf{y}} [h_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x})]^2 p(\mathbf{x}, \mathbf{y}) d\mathbf{x} d\mathbf{y} P(\mathcal{D}) d\mathcal{D}}_{\text{VARIANCE: error due to training dataset}}$$



Bias-Variance Trade-off: Illustration

How can we reduce the variance?

- Use a lot of data (ie, increase the size of $\mathcal{D}$)
- Use a simple $h(\cdot)$ so that $h_{\mathcal{D}}(\mathbf{x})$ does not vary much across different training datasets. An extreme example is $h(\mathbf{x}) = \text{const}$.



The remaining item

$$\begin{aligned}\mathbb{E}_{\mathcal{D}} R[h_{\mathcal{D}}(\mathbf{x})] &= \int_{\mathcal{D}} \int_{\mathbf{x}} \int_y [h_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x})]^2 p(\mathbf{x}, y) d\mathbf{x} dy P(\mathcal{D}) d\mathcal{D} \\ &\quad + \int_{\mathcal{D}} \int_{\mathbf{x}} \int_y [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x}) - y]^2 p(\mathbf{x}, y) d\mathbf{x} dy P(\mathcal{D}) d\mathcal{D}\end{aligned}$$

The integrand has no dependency on $\mathcal{D}$ anymore and simplifies to

$$\int_{\mathbf{x}} \int_y [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x}) - y]^2 p(\mathbf{x}, y) d\mathbf{x} dy$$

We will apply a similar add-and-subtract trick, by using an averaged target y (what we want to predict from $\mathbf{x}$):

$$\mathbb{E}_y[y|\mathbf{x}] = \int_y y p(y|\mathbf{x}) dy$$

Decompose again

$$\begin{aligned} & \int_{\mathbf{x}} \int_y [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x}) - y]^2 p(\mathbf{x}, y) d\mathbf{x} dy \\ &= \int_{\mathbf{x}} \int_y [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_y[y|\mathbf{x}] + \mathbb{E}_y[y|\mathbf{x}] - y]^2 p(\mathbf{x}, y) d\mathbf{x} dy \\ &= \underbrace{\int_{\mathbf{x}} \int_y [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_y[y|\mathbf{x}]]^2 p(\mathbf{x}, y) d\mathbf{x} dy}_{\text{BIAS}^2} \\ &\quad + \underbrace{\int_{\mathbf{x}} \int_y [\mathbb{E}_y[y|\mathbf{x}] - y]^2 p(\mathbf{x}, y) d\mathbf{x} dy}_{\text{NOISE}} \end{aligned}$$

Where is the cross-term?

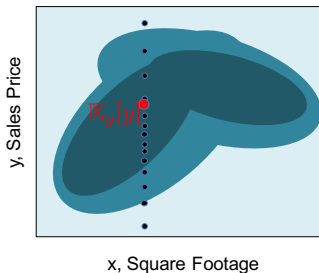
Take-home exercise: Show that it is zero

Bias-Variance Trade-off: Illustration

- For a given $\mathbf{x}$, we have a conditional distribution $p(y|\mathbf{x})$ of sales prices; the Bayesian optimal prediction of the label value for a given $\mathbf{x}$ is $\mathbb{E}_y[y|\mathbf{x}]$;
- The noise term measures the inherent variance in labels y

$$\underbrace{\int_{\mathbf{x}} \int_y [\mathbb{E}_y[y|\mathbf{x}] - y]^2 p(\mathbf{x}, y) d\mathbf{x} dy}_{\text{NOISE: error due to randomness of } y}$$

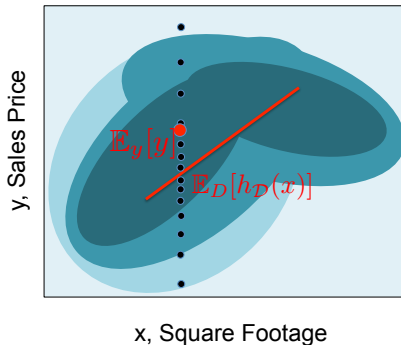
- This term has nothing to do with our prediction $h_{\mathcal{D}}(\mathbf{x})$



Bias-Variance Trade-off: Illustration

- If our model class was rich enough (eg. a high-degree polynomial), then $\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x})$ should perfectly match $\mathbb{E}_y[y|\mathbf{x}]$
- Restricting to simpler models (eg. linear) results in a bias

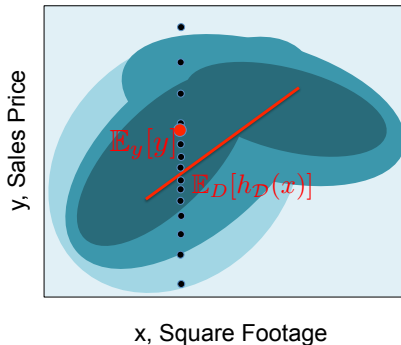
$$\underbrace{\int_{\mathbf{x}} \int_y [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_y[y|\mathbf{x}]]^2 p(\mathbf{x}, y) d\mathbf{x} dy}_{\text{BIAS}^2: \text{error due to the model approximation}}$$



Bias-Variance Trade-off: Illustration

How can we reduce the bias?

It can be reduced by using more complex models. We shall choose $h(\cdot)$ to be as flexible as possible: the better $h(\cdot)$ approximates $\mathbb{E}_y[y|\mathbf{x}]$, the smaller the bias. However, this will increase the VARIANCE term.



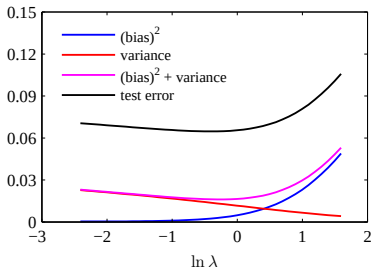
Bias/variance tradeoff

Error decomposes into 3 terms

$$\mathbb{E}_{\mathcal{D}} R[h_{\mathcal{D}}(\mathbf{x})] = \text{VARIANCE} + \text{BIAS}^2 + \text{NOISE}$$

where the first and the second term are inherently in conflict in terms of choosing what kind of $h(\mathbf{x})$ we should use (unless we have an infinite amount of data).

If we can compute all terms analytically, they will look like this



Summary of risk components

The average risk (with quadratic loss) can be decomposed as:

$$\begin{aligned}\mathbb{E}_{\mathcal{D}} R[h_{\mathcal{D}}(\mathbf{x})] &= \underbrace{\int_{\mathcal{D}} \int_{\mathbf{x}} \int_y [h_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x})]^2 p(\mathbf{x}, y) d\mathbf{x} dy P(\mathcal{D}) d\mathcal{D}}_{\text{VARIANCE: error due to training dataset}} \\ &+ \underbrace{\int_{\mathbf{x}} \int_y [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_y[y|\mathbf{x}]]^2 p(\mathbf{x}, y) d\mathbf{x} dy}_{\text{BIAS}^2: \text{error due to the model approximation}} \\ &+ \underbrace{\int_{\mathbf{x}} \int_y [\mathbb{E}_y[y|\mathbf{x}] - y]^2 p(\mathbf{x}, y) d\mathbf{x} dy}_{\text{NOISE: error due to randomness of } y}\end{aligned}$$

Here we define: $h_{\mathcal{D}}(\mathbf{x})$ as the output of the model trained on $\mathcal{D}$, $\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x})$ as the expectation of the model over all datasets $\mathcal{D}$, and $\mathbb{E}_y[y|\mathbf{x}]$ as the expected value of y .

Example: Why regularized linear regression could be helpful?

Model

$$h(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$$

Consider the best possible (linear) $h^*(\mathbf{x})$

$$\mathbf{w}^* = \operatorname{argmin}_{\mathbf{w}} \int_{\mathbf{x}} [\mathbb{E}_y[y|\mathbf{x}] - \mathbf{w}^\top \mathbf{x}]^2 p(\mathbf{x}) d\mathbf{x}$$

Note that this linear model assumes the knowledge of joint distribution, thus, not achievable. Intuitively, it is the *best* linear model that can predict the data most accurately.

More refined decomposition of the bias

$$\begin{aligned}\int_{\mathbf{x}} [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_y[y|\mathbf{x}]]^2 p(\mathbf{x}) d\mathbf{x} &= \int_{\mathbf{x}} [h^*(\mathbf{x}) - \mathbb{E}_y[y|\mathbf{x}]]^2 p(\mathbf{x}) d\mathbf{x} \\ &+ \int_{\mathbf{x}} [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x}) - h^*(\mathbf{x})]^2 p(\mathbf{x}) d\mathbf{x}\end{aligned}$$

- **First term – Model bias:** the price we pay for choosing linear functions to model data. This is the difference between the prediction of the best possible linear model and the actual target.
- **Second term – Estimation bias:** the difference between the optimal model and the estimated model.

Normally, the estimation bias is zero if we do not regularize.

Bias/variance tradeoff for regularized linear regression

We can only adjust estimation bias

$$\int_{\mathbf{x}} [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\mathbf{x}; \lambda) - h^*(\mathbf{x})]^2 p(\mathbf{x}) d\mathbf{x}$$

where $h(\mathbf{x}; \lambda)$ is the estimated model with regularized linear regression (parameterized with λ).

This term will not be zero anymore!

Thus, bias goes up.

But, as long as this is balanced with a decrease in variance, we are willing to do so.

1. Review of Ridge Regression
2. Non-linear Basis Functions
3. Overfitting and Regularization
4. Hyperparameter Tuning and Cross-Validation
5. Bias-Variance Trade-off